

就活スケジュール管理システムの設計と実装

— 予定の重複を信号機カラーで可視化し、面接の振り返りを次に活かす Web アプリケーション —

氏名 (学籍番号: k24032kk)

愛知工業大学 情報科学部 / データベース 2026 期末レポート

概要. 本稿では、就職活動における面接・インターン・説明会などの予定を一元管理し、**予定の時間的な重複や移動リスクを信号機カラー（赤・黄・青）で可視化**する Web アプリケーションの設計と実装について述べる。就職活動では複数企業の選考が同時並行で進むため、日程の重複（ダブルブッキング）やエントリーシート（ES）の提出漏れといった「本来防げるミス」で機会を失う学生が少なくない。本システムは、PHP のフレームワーク Laravel と MySQL を用いたサーバサイドレンダリング型の Web アプリケーションとして実装し、実行環境を Docker で統一した。予定どうしの重複は区間の交差判定に基づき、重複なしでも前後の間隔が短い場合は移動リスクとして黄色で警告する。さらに、面接で実際にされた質問とその手応えを記録し、**うまく答えられなかった質問を全企業横断でリスト化**することで次の面接に活かせるようにした。加えて、就活生の悩みに関する調査結果に基づき、志望度・締切アラート・選考フェーズ管理を追加した。開発では、テスト駆動と、ブラウザを自動操作する「評価エージェント」による PDCA サイクルを取り入れ、反復的に品質を高めた。

1. はじめに

就職活動（以下、就活）では、説明会・エントリーシート（ES）提出・複数回の面接といった多様な予定が、複数の企業について同時並行で進行する。一般に学生は十数社規模の企業に応募するため、予定は短期間に集中し、**面接日程の重複やES提出期限の見落とし**といった、注意さえ払えば防げるはずのミスによって貴重な選考機会を失うことがある。就活情報サイトの調査でも、就活生の悩みとして「スケジュールリングがうまくできない」「焦りや不安」が上位に挙げられており [1]、情報量と締切の多さへの対処が大きな課題であることが分かる。

一方で、面接そのものについても課題がある。面接は「やりっぱなし」にせず、質問の意図に対して的確に答えられたかを振り返り、改善点を次の対策に活かすことが推奨される [2]。しかし多くの学生は、振り返りを記録として残す手段を持たず、同じ種類の質問に再び詰まってしまう。

本研究では、これらの課題に対し、**(1) 予定の重複・移動リスクの可視化**と**(2) 面接の振り返りの蓄積と横断的な活用**を中核に据えた就活スケジュール管理システムを設計・実装した。本システムの新規性は、単なるカレンダーやタスク管理に留まらず、予定間の関係（重複・間隔）を**信号機カラー**として直感的に提示する点、および答えられなかった質問を企業をまたいで集約し「**苦手質問リスト**」として提示する点にある。本稿では、関連する既存手段との比較（2 章）、システムの概要（3 章）、設計の詳細（4 章）、実装（5 章）、開発手法と評価エージェントによる改善（6-7 章）、考察（8 章）の順に述べる。

2. 関連する既存の手段と本システムの位置づけ

就活生が予定管理に用いる既存の手段は、大きく三つに分けられる。第 1 は表計算ソフトによる管理で、企業名・締

切・選考フェーズを表に整理する方法である。一覧性は高いが、予定どうしの「時間の重なり」を自動では検出できず、目視に頼るため見落としが生じやすい。第 2 は汎用カレンダーアプリで、予定を時間軸上に配置できるが、個々の予定を表示するにとどまり、予定間の関係（重複や間隔の長さ）を強調する仕組みは持たない。第 3 はノート系ツールや就活専用アプリで、企業ごとの選考状況を管理できるものの、面接で実際にされた質問を蓄積し「**答えられなかったものだけ**」を横断的に抽出する機能は一般的でない。

本システムは、これらに対し、(a) 予定間の関係を信号機カラーとして明示する点、(b) 面接の振り返りを質問単位で蓄積し、手応えに基づいて苦手質問を横断抽出する点、(c) 重複時の優先判断を支える志望度を併せ持つ点で差別化される。すなわち「**予定の管理**」と「**面接経験の活用**」を一体化し、就活特有の課題に焦点を絞っている点に本システムの位置づけがある。

3. システムの概要

3.1 目的と対象利用者

本システムの目的は、就活中の学生が自身の予定と選考状況を一元管理し、予定の衝突を未然に防ぎ、面接経験を次に活かせるようにすることである。対象利用者は就活中の大学生であり、各自がアカウントを登録してログインする**マルチユーザー方式**を採用。データは利用者ごとに完全に分離され、他者の予定や企業情報は一切見えない。

3.2 主な機能

本システムは以下の機能を提供する。

1. 認証: 利用者の登録・ログイン・ログアウト。
2. 企業管理: 応募先企業の登録・編集・削除、選考状況（フェーズ）・志望度・メモを保持。

3. 予定管理：面接・説明会・ES 締切などの予定を、Google カレンダー風の UI で登録・編集・削除。
4. 被り検出：予定どうしの時間的な重複（赤），および移動リスク（黄）を信号機カラーで可視化。
5. 面接の振り返り：各面接に「実際にされた質問・手応え・次はこう答えるメモ」を記録。
6. 苦手質問リスト：手応えが「答えられず」の質問を全企業横断で一覧化。
7. ダッシュボード：直近予定・重複警告・締切アラートを集約表示。

3.3 画面構成

画面は、ログイン／登録、ダッシュボード、カレンダー（メイン）、企業一覧・詳細・登録／編集、予定登録／編集／詳細、苦手質問リストから構成される。カレンダー画面では、後述する被り検出の結果に応じて各予定が色分け表示され、凡例（赤＝重複、黄＝間隔が短い、青＝通常）を併記している。画面間の遷移はダッシュボードを起点とし、グローバルナビゲーション（ホーム・カレンダー・企業・苦手質問）から各機能へ到達できる。主要な画面遷移を図 1 に示す。カレンダー上の予定や企業一覧の各項目はクリックで詳細画面へ遷移し、詳細画面から編集画面へ進める構造とした。これにより、利用者は「一覧で全体を俯瞰し、気になる項目の詳細へ掘り下げる」という自然な操作で目的の情報に到達できる。

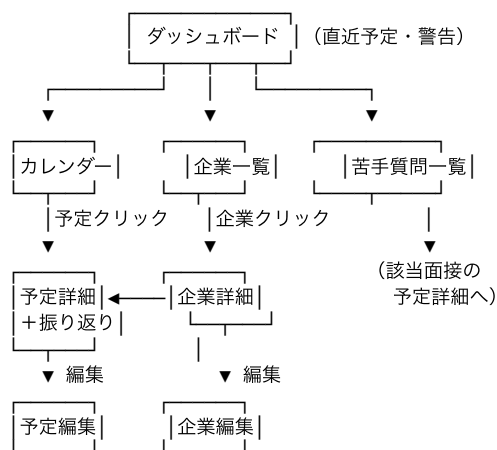


図 1 主要な画面遷移図（一覧→詳細→編集の三層構造）

3.4 想定利用シナリオ

本システムの典型的な利用の流れは次のとおりである。(1) 利用者はまず応募する企業を登録し、志望度と選考状況を設定する。(2) 企業から面接や説明会の連絡が来るたびに、その日時を予定として登録する。(3) 新たな予定を登録した際、既存の予定と時間が重なれば、カレンダーとダッシュボードに赤色の警告が表示される。利用者は併記された志望度を見て、どちらを優先し、どちらの日程変更を依頼するかを判断する。(4) 面接を終えたら、その予定の詳細画面で、実際にされた質問と手応え、次回への改善メモを記録する。(5) 次の面

接の前には「苦手質問リスト」を開き、過去に答えられなかった質問を企業横断で見直して準備する。このように、本システムは就活の「予定の管理」と「経験の蓄積」を一つの流れの中で支援する。

4. システムの設計

4.1 全体アーキテクチャ

本システムは、PHP のフレームワーク Laravel を用いた Blade テンプレートによるサーバサイドレンダリング（SSR）型のモノリス構成として設計した。API+SPA 構成は本課題には過剰であると判断し、採用していない。処理の流れは、ルーティング → コントローラ → モデル（Eloquent ORM）→ ビュー（Blade）という Model-View-Controller（MVC）パターンに従う（図 2）。唯一、カレンダー画面のみ、動的な月／週表示のために JavaScript ライブラリ FullCalendar.js を用い、サーバから JSON で受け取った予定を描画する。

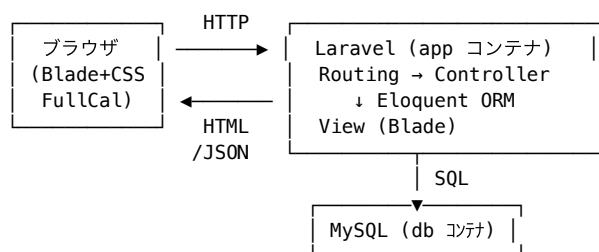


図 2 全体アーキテクチャ（Docker 上の 2 コンテナ構成）

実行環境は Docker（Laravel Sail）で統一し、アプリケーション（PHP/Laravel）と MySQL の 2 コンテナで構成した。これにより、利用者のマシンに PHP や MySQL を直接導入することなく、同一の環境を再現できる。データベースの接続ユーザには、要件に従い root ではない一般ユーザ（sail）を用いている。

4.2 データベース設計

本システムのデータモデルは、利用者（users）、企業（companies）、予定（events）、面接質問（interview_questions）の 4 テーブルからなる。エンティティ間の関係を図 3 に示す。1 人の利用者は複数の企業・予定・質問を持ち（1 対多）、1 つの企業は複数の予定を持ち、1 つの予定は複数の面接質問を持つ。

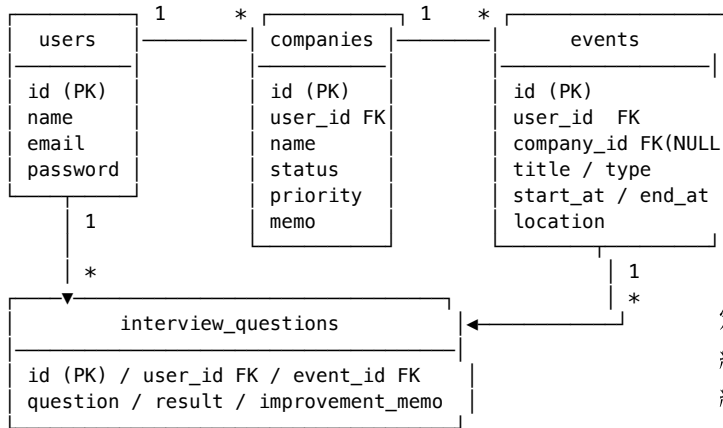


図3 エンティティ関係 (ER) 図

各テーブルの主な属性を表1に示す。すべての主要テーブルは `user_id` を外部キーとして持ち、アプリケーション層のすべての検索クエリに `where(user_id = ログイン中の利用者)` のスコープを適用することで、マルチユーザ環境におけるデータの分離を保証している。参照整合性は外部キー制約で担保し、利用者削除時には関連データが連鎖削除 (cascade) される。

テーブル設計は第3正規形を満たすよう正規化しており、企業と予定、予定と質問はそれぞれ独立したテーブルに分割して多重度 (1 対多) を表現した。予定の企業参照 (`company_id`) は、企業に紐付かない予定 (例: 合同説明会) も登録できるよう NULL を許容する。外部キー列には索引が自動的に付与されるため、「ある利用者の予定一覧」「ある企業の関連予定」といった頻出の絞り込みが効率的に実行される。日時は `DATETIME` 型で保持し、アプリケーションのタイムゾーンを Asia/Tokyo に統一することで、保存・表示・カレンダー描画の間で時刻の不整合が生じないようにした。

表1 主要テーブルの構造

テーブル	主な属性 (型)
users	id, name, email, password (Breeze 標準)
companies	id, user_id, name, status (選考フェーズ), priority (志望度 1-3), memo
events	id, user_id, company_id?, title, type (面接/説明会/ES締切/その他), start_at, end_at, location
interview_questions	id, user_id, event_id, question, result (good/ok/bad), improvement_memo

4.3 予定の被り検出アルゴリズム

本システムの中核は、予定どうしの「被り」を検出し信号機カラーを割り当てる処理である。これを画面やデータベースから独立した純粋なロジック (`ConflictDetector` クラス) として分離し、単体テスト可能にした。各予定には開始時刻 s と終了時刻 e があり、2 予定 A, B に対して次のように状態を定める。

- 赤 (重複): 区間が交差するとき。すなわち $A.s < B.e \wedge B.s < A.e$ 。
- 黄 (移動リスク): 重複はしないが、先に終わる予定の終了から次の予定の開始までの間隔が閾値 (既定 60 分) 未満のとき。
- 青 (通常): 上記いずれにも該当しないとき。

状態の優先順位は 赤 > 黄 > 青 とし、ある予定が複数の予定と関係を持つ場合は最も重大な状態を採用する。2 予定の組に対する状態決定の流れを図4に示す。ここで g は「先に終わる予定の終了から次の予定の開始までの間隔」、 T は移動リスクの閾値 (既定 $T = 60$ 分) を表す変数である。

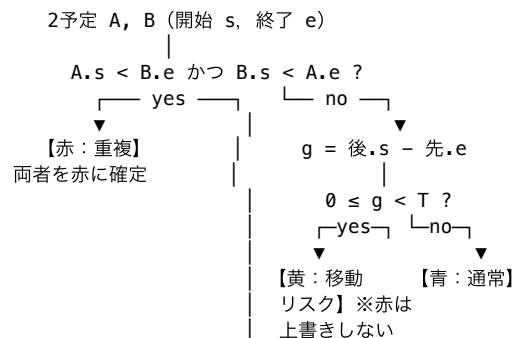


図4 2 予定の組に対する被り状態の決定フロー (変数 g : 予定間の間隔, T : 閾値60分)

全予定の各組についてこの判定を行い、各予定の最終状態を求める。計算量は予定数 n に対し全対比較で $O(n^2)$ であるが、個人が一度に把握する予定の規模では十分高速である。この処理を画面・データベースから独立した純粋なクラス `ConflictDetector` として実装した点が設計上の要であり、これにより境界条件 (重複・間隔がちょうど閾値 T ・優先順位) を単体テストで厳密に検証できる。実装の詳細は付録Aに示す。

4.4 面接の振り返りと苦手質問の横断抽出

面接の予定には、複数の「面接質問」を記録できる。各質問は、本文、手応え (good=うまく/ok=微妙/bad=答えられず)、および「次はこう答える」改善メモを持つ。苦手質問リストは、ログイン中の利用者の質問のうち `result = bad` のものを、企業名・面接日とともに新しい順で抽出した横断ビューである。これにより、次の面接の直前に、企業をまたいで「詰まった質問」だけを効率的に見直せる。

4.5 就活生の課題に基づく機能拡張

就活生の悩みに関する調査 [1][3] を踏まえ、次の3機能を追加した。第1に、日程が重複した際に「どちらを優先するか」の判断材料として企業に志望度 (第一志望/志望/興味あり) を持たせ、重複警告に併記した。第2に、ESの提出漏れを防ぐため、ダッシュボードに7日以内に迫る予定の締切アラートを、重複警告と同時に、予定の具体名つきで表示した。第3に、持ち駒 (選考中企業) の管理を容易にするため、企業の選考状況をエントリーから内定までの8段階の選

考フェーズとして定型化し、フェーズの進行が一目で分かるよう配色した。選考フェーズが取り得る状態とその遷移を図 5 に示す。各企業はいずれか一つの状態を持ち、選考の進行に応じて利用者が状態を更新する。いずれの段階からも「お祈り」（不採用）へ遷移し得る点が、採用プロセスの特徴である。

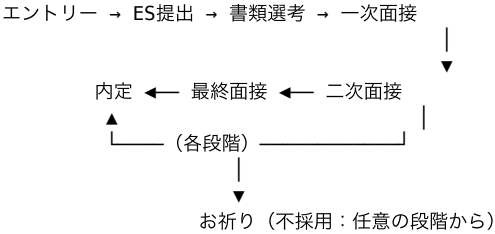


図 5 選考フェーズの状態遷移（各状態は `companies.status` が取る値に対応）

5. 実装

5.1 開発・実行環境

実装は PHP / Laravel, MySQL, Tailwind CSS, FullCalendar.js を用い、認証には Laravel Breeze を採用した。実行環境は Docker (Laravel Sail) で、docker compose によりアプリと MySQL の 2 コンテナを起動する。データベースの初期化は、スキーマを定義するマイグレーションと、初期データを投入するシードからなる。シードには、デモ用利用者に加え、赤・黄・青の 3 状態と ES 締切アラートを確認できるサンプル予定を投入するコードを含めた。DB ユーザは root 以外の一般ユーザ sail である。初期化スクリプト（スキーマ定義と初期データ投入）の主要部を付録 B に示す。

5.2 主要機能の実装

企業・予定は Laravel のリソースコントローラにより CRUD を実装し、すべての操作で所有者検証（他人のデータへのアクセスは HTTP 403）を行う。カレンダー画面は、サーバ側で全予定に対し被り検出を実行し、各予定に色（赤 `#ef4444`, 黄 `#eab308`, 青 `#3b82f6`）を割り当てた JSON を `/calendar/events` から配信し、FullCalendar.js が描画する。被り検出の結果はカレンダーだけでなく、ダッシュボードや企業詳細の予定一覧にも「信号機ドット」として展開し、どの画面からでも重複に気づけるようにした。モバイル幅では、月表示では各セルが狭く可読性が低いため、リスト表示へ自動的に切り替える。

5.3 カレンダーとサーバの連携

カレンダー画面の描画は、HTML の初期表示と予定データの取得を分離した設計とした。利用者がカレンダー画面を開くと、まず予定を含まない骨組みの HTML が返され、描画用の JavaScript (FullCalendar.js) が初期化される。続いて JavaScript が `/calendar/events` へ非同期的に要求を送り、サーバはログイン中の利用者の全予定に対し被り検出を実行し

たうえで、各予定に「開始・終了・タイトル・色・詳細への遷移先」を付与した JSON を返す。色は被り状態（赤・黄・青）を 16 進カラーコードに対応づけた値である。この分離により、月や週の表示を切り替えても画面全体を再読み込みすることなく、必要なデータだけを取得して再描画できる。被り状態の計算をサーバ側に集約したことで、カレンダー・ダッシュボード・企業詳細のいずれの画面でも同一の判定結果を一貫して用いることができる。

5.4 データ分離とセキュリティ

マルチユーザ環境では、ある利用者のデータが他者に漏れないことが必須要件である。本システムでは、企業・予定・面接質問の各テーブルが `user_id` を持ち、一覧取得・詳細表示・更新・削除のすべての操作において「ログイン中の利用者の `user_id` に一致するか」を確認する。具体的には、一覧取得は `user_id` による絞り込みを行い、個別データへの操作は所有者でなければ HTTP 403 (Forbidden) を返す。この所有者検証が漏れると重大な情報漏えいにつながるため、後述するように「他人のデータが見えない／操作できない」ことをフィーチャテストで明示的に検証している。パスワードはフレームワークの認証機構によりハッシュ化して保存し、フォーム送信には CSRF トークンを付与して、なりすまし投稿を防いでいる。

5.5 テスト

品質保証として PHPUnit による自動テストを整備し、合計 41 件 (97 アサーション) が全て成功する状態を維持した。内訳は、被り検出ロジックの境界条件を検証する単体テスト、および各機能の振る舞いと「他人のデータが見えない」ことを検証する結合（フィーチャ）テストである。表 2 に主なテスト観点を示す。特に、各機能のテストには「他人のデータにアクセスできない」ことを確認する観点を必ず含め、マルチユーザ環境で最も重大なデータ漏えいの不具合を機械的に検出できるようにした。データベースを伴うテストでは、各テストの実行ごとにスキーマを再構築して独立性を担保している。これにより、機能追加や改修のたびに全テストを実行することで、既存機能を壊していないこと（回帰のないこと）を継続的に確認できる。

表 2 主なテスト観点

対象	検証内容
被り検出	重複＝赤, 間隔30分＝黄, 間隔120分＝青, 赤>黄の優先順位
企業/予定	作成・所有者検証・他人のデータ非表示
カレンダー	重複時に赤色を返す・他人の予定を除外
振り返り	質問の登録・他人の面接への登録拒否
苦手質問	bad のみ抽出・他人の質問を非表示
初期化	シードがデモデータを生成

6. 開発手法

本システムは、行き当たりばつたりの実装を避けるため、「設計 → 計画 → 実装」の段階を明確に分けて開発した。まず、解決したい課題と機能の範囲、データモデル、被り検出の定義を文書化した設計書を作成し、作るもの・作らないものを確定した。次に、設計書をもとに、足場づくり・認証・企業管理・予定管理・被り検出・カレンダー・面接振り返り・苦手質問リストといった単位に作業を分解した実装計画を作成した。各単位は、それ単独で動作し検証できる粒度とした。

実装は、テスト駆動開発（TDD）の方針で進めた。各機能について、先に「期待する振る舞い」を検証するテストを書き、それが失敗することを確認してから最小限の実装を行い、テストが成功することを確認して次へ進む、という手順を繰り返した。特に被り検出ロジックは、重複・間隔ちょうど閾値・優先順位といった境界条件をテストで先に定義することで、仕様の曖昧さを排除した。作業の単位ごとに動作とテスト成功を確認してからコミットすることで、どの時点でも「動く状態」を保ちながら開発を進められた。こうした段階的かつ検証可能な進め方は、後述する評価エージェントによる反復的改善と組み合わせることで、限られた期間でも品質を確保する基盤となった。

7. 開発プロセスの工夫：評価エージェントによるPDCA

本開発では、実装の品質を反復的に高めるため、ブラウザを自動操作する「評価エージェント」を用いた PDCA サイクルを取り入れた。具体的には、(Plan) 改善点を特定し、(Do) 画面を実装・修正し、(Check) 評価エージェントが実際に起動中のアプリへログインして各画面を巡回・スクリーンショットを取得し、可読性・操作性・主要機能の妥当性を評価して優先度付きの指摘を返し、(Act) その指摘を反映する、という流れを複数サイクル繰り返した。

各サイクルで評価エージェントが指摘した代表的な問題と、それに対する是正を表 3 に示す。

表 3 評価エージェントによる PDCA の主な是正内容

サイクル	検出された問題	是正 (Act)
1	主要ボタンが背景と同化し不可視／全体が素っ気ない	配色・余白・カード化で再設計
1	カレンダーの時刻が 9 時間ずれて表示	タイムゾーンを Asia/Tokyo に統一
1	被りカラーがカレンダーにしか出ない	ダッシュボード・企業詳細にも展開
2	モバイルでカレンダーが判読不能	モバイルはリスト表示に自動切替
3	重複時にどちらを優先すべきか不明	志望度を追加し警告に併記

4	締切警告が直前すぎる／具体名がない	7 日前から具体名つきで通知
---	-------------------	----------------

このプロセスにより、例えば「カレンダー上の時刻が実際より 9 時間ずれて表示される」というタイムゾーン起因の重大な不具合や、主機能である被りカラーがカレンダー以外の画面に現れていない問題などを早期に発見・修正できた。人手のレビューを模した自動評価を開発ループに組み込むことで、機能追加のたびに「見やすさ」と「主機能の一貫性」を検証でき、課題提出物としての完成度を体系的に高められた点が本開発の特徴である。

8. 考察と今後の課題

本システムは、予定間の関係を信号機カラーとして提示することで、従来のカレンダーが「点（個々の予定）」しか示さなかったのに対し、「関係（重複・間隔）」を可視化できた。また、被り検出を純粹ロジックとして分離した設計は、単体テストによる厳密な検証を可能にし、保守性の面でも有効であった。志望度や締切アラートの追加は、就活生の「どちらを優先するか」「締切を忘れない」という現実の悩みに直接対応している。

設計上のトレードオフについても述べる。本システムでは、画面遷移のたびにサーバが HTML を生成する Blade (SSR) 方式を採用し、API+SPA 方式を見送った。SSR は実装が単純で、フレームワークの認証やフォーム処理をそのまま活用できる利点がある一方、画面の部分更新には不向きである。本システムでは、動的描画が本質的に必要なカレンダーに限って JavaScript (FullCalendar.js) を併用し、それ以外は SSR に統一することで、実装の単純さと操作性を両立させた。被り検出を全対比較 ($O(n^2)$) とした点も、個人が扱う予定数では性能上の問題が生じないため、アルゴリズムの単純さと正しさを優先した判断である。

本システムの限界として、第 1 に、予定間の被りは「時間」のみで判定しており、オンラインと対面の別や会場間の距離といった物理的な移動可能性は考慮していない。移動リスク（黄）の閾値も一律 60 分の固定値であり、個々の事情に合わせた調整はできない。第 2 に、提出漏れ防止のための締切アラートはアプリを開いたときにのみ表示される「プル型」であり、利用者がアプリを開かなければ気づけない。能動的に通知する「プッシュ型」のリマインドは本課題の範囲外とした。これらは、信号機カラーによる可視化という本システムの中心的価値を損なわない範囲での簡潔さを優先した結果である。

今後の課題として、(1) 志望度順の並び替えや選考状況による絞り込み、(2) ES 締切のカレンダー上での専用表現、(3) 被り検出の計算量を区間ソートにより $O(n \log n)$ へ改善すること、(4) 移動手段や会場を考慮した移動リスク判定、が挙げられる。通知メールや外部カレンダー連携も、実用化に際しては検討に値する。

9. おわりに

本稿では、予定の重複を信号機カラーで可視化し、面接の振り返りを横断的に活用できる就活スケジュール管理システムの設計と実装を述べた。Laravel と MySQL を使い、Docker で実行環境を統一し、被り検出ロジックの分離・自動テスト・評価エージェントによる PDCA といった工夫により、動作し、かつ保守可能なシステムを構築した。本システムは、就活生が「防げるミス」を避け、面接経験を次に活かすための実用的な支援ツールとなることを目指している。

参考文献

1. 就活の教科書編集部：就活生の悩みランキング TOP10, <https://reashu.com/shukatsusei-nayami/> (2026 年 6 月閲覧)。
2. 女の転職 type：失敗を糧にする！面接の振り返り方法, <https://woman-type.jp/academia/knowhow/interview/review/> (2026 年 6 月閲覧)。
3. ホワイト企業就活：就活管理アプリに求められる機能, <https://jws-japan.or.jp/whitecareer/blog/5632> (2026 年 6 月閲覧)。

付録 A 被り検出ロジック (ConflictDetector)

本文 3.3 節で述べた被り検出の中核実装を以下に示す (PHP)。入力 は各予定の id・開始・終了からなる配列、出力は予定 id から状態 (赤/黄/青) への対応である。

```
class ConflictDetector {
    public function __construct(private int $T = 60) {}

    // $events: [['id'=>, 'start'=>Carbon, 'end'=>Carbon], ...]
    public function detect(array $events): array {
        $status = [];
        foreach ($events as $e) $status[$e['id']] = 'normal';

        $n = count($events);
        for ($i = 0; $i < $n; $i++) {
            for ($j = $i + 1; $j < $n; $j++) {
                $a = $events[$i]; $b = $events[$j];
                // 重複 (赤) : a.s < b.e かつ b.s < a.e
                if ($a['start']->lt($b['end'])
                    && $b['start']->lt($a['end'])) {
                    $status[$a['id']] = $status[$b['id']] = 'red';
                    continue;
                }
                // 間隔 (移動リスク・黄)
                [$prev, $next] = $a['start']->lte($b['start'])
                    ? [$a, $b] : [$b, $a];
                $g = $prev['end']->diffInMinutes($next['start'], false);
                if ($g >= 0 && $g < $this->T) {
                    if ($status[$prev['id']] !== 'red')
                        $status[$prev['id']] = 'yellow'; // 赤は上書きしない
                    if ($status[$next['id']] !== 'red')
                        $status[$next['id']] = 'yellow';
                }
            }
        }
        return $status;
    }
}
```

付録 B 初期化スクリプト (抜粋)

スキーマ定義 (マイグレーション) と初期データ投入 (シーダ) の要点を示す。重複 (赤)・移動リスク (黄)を確認できるデモ予定を投入している。

```
// マイグレーション: events テーブル
Schema::create('events', function (Blueprint $t) {
    $t->id();
    $t->foreignId('user_id')->constrained()-
>cascadeOnDelete();
    $t->foreignId('company_id')->nullable()
    ->constrained()->nullOnDelete();
    $t->string('title');    $t->string('type')-
>default('面接');
    $t->dateTime('start_at'); $t->dateTime('end_at');
    $t->string('location')->nullable();    $t-
>timestamps();
});

// シーダ: 重複(赤)のデモ予定
Event::create(['user_id'=>$u->id, 'company_id'=>$a-
>id,
    'title'=>'A社 最終面接', 'type'=>'面接',
    'start_at'=>'2026-06-15 14:00', 'end_at'=>'2026-06-
15 15:00']);
Event::create(['user_id'=>$u->id, 'company_id'=>$b-
>id,
    'title'=>'B社 説明会', 'type'=>'説明会',
    'start_at'=>'2026-06-15 14:30', 'end_at'=>'2026-06-
15 15:30']);
```